

TP2 — Algoritmos Genéticos

SIA-G3 (nombre del equipo pendiente)

2026

Ejercicio 2 — Resumen

Implementamos, íntegramente a mano y sin ninguna librería de Algoritmos Genéticos, un motor de AG que aproxima una imagen objetivo componiendo triángulos translúcidos sobre un canvas en blanco. El único input real del problema (sin contar los hiperparámetros) es la imagen a aproximar y la cantidad máxima de triángulos disponibles; el cromosoma, el operador de selección, la cruce, la mutación y el criterio de corte son todos configurables desde un único archivo JSON, sin tocar código. El resultado no se defiende “porque funciona”, sino con una matriz de 75 corridas medidas, con 5 semillas por celda, que compara cuantitativamente cada familia de operadores estudiada en la cátedra.

Estructura del cromosoma

Cada individuo es un vector plano de float32 de longitud $11 * \text{budget}$, donde `budget` es la cantidad máxima de triángulos pedida por el usuario. `GENES_PER_TRIANGLE = 11` fija el layout de cada triángulo: `X1, Y1, X2, Y2, X3, Y3, R, G, B, A, ACTIVE` — tres vértices, un color RGBA y un flag de actividad, todos normalizados a $[0,1]$. El flag `ACTIVE` (umbral en 0.5) es lo que permite que la cantidad *efectiva* de triángulos varíe sin necesitar cromosomas de longitud variable: la longitud del vector es fija (y por lo tanto los cuatro operadores de cruce — un punto, dos puntos, uniforme, anular — quedan definidos exactamente como se los vio en clase), pero un triángulo con `ACTIVE < 0.5` no se dibuja y no cuenta para el presupuesto usado.

Los vértices están acotados a una banda de sobrepaso $[-0.1, 1.1]$ (para que un triángulo pueda cubrir una esquina del canvas sin que sus tres vértices tengan que caer exactamente sobre el borde) y el canal alfa a $[0.1, 0.9]$ (alfa 0 desperdicia presupuesto de triángulos, alfa 1 mata la translucidez que pide el enunciado). El único punto de reparación definido en todo el motor es `reflect()`: cuando una mutación empuja un gen fuera de su banda, se lo refleja como una onda triangular en vez de recortarlo (`clip`). Elegimos reflejar y no recortar porque recortar amontona probabilidad sobre el límite superior de la banda — termina el lado del triángulo demasiado cerca del borde

Función de fitness

El fitness de un individuo es $\text{fitness} = \max(1 - \sqrt{\text{SSE} / \text{SSE}_{\text{max}}}, \text{FITNESS_FLOOR})$, donde SSE es la suma de errores cuadráticos por píxel entre el render del individuo y la imagen objetivo, y SSE_max es el SSE del peor caso posible sobre ese mismo canvas. Es, en esencia, un RMSE normalizado e invertido: 1.0 para una réplica exacta, decreciente a medida que el error crece. FITNESS_FLOOR (una constante estrictamente positiva, del orden de $1e-12$) evita que el fitness llegue exactamente a cero: los métodos de selección de la familia ruleta (ruleta, universal, ranking, Boltzmann) necesitan que la suma de fitness sobre la población sea estrictamente positiva para que la probabilidad acumulada esté bien definida, y sin el piso un individuo maximamente incorrecto rompería esa cuenta.

Deliberadamente, la función de fitness **no** penaliza la cantidad de triángulos activos. Esto no es un olvido: fue una decisión explícita ya en la Fase 1 del proyecto (assumption A-02). El motivo es doble. Primero, el enunciado especifica el fitness como error por píxel, sin mencionar ningún término de tamaño. Segundo, y más importante, agregar una penalización por cantidad de triángulos activos confundiría exactamente la comparación que este proyecto está armado para medir: si dos configuraciones de selección terminan con una distinta cantidad de triángulos activos, ¿se termina la

Preguntas previas al experimento

El enunciado exige responder ocho preguntas concretas antes de empezar a experimentar. Las respondemos acá, cada una apoyada en algo que realmente construimos y medimos — no como una reformulación de la pregunta.

¿Cómo evalúo mi aproximación al dibujo?

Con la función de fitness ya descripta: $\text{fitness} = \max(1 - \sqrt{\text{SSE} / \text{SSE_max}}, \text{FITNESS_FLOOR})$, un RMSE normalizado e invertido sobre los tres canales de color de cada píxel. Es una métrica continua, estrictamente positiva y con un techo teórico de 1.0, lo que la hace directamente comparable entre corridas con distinta imagen objetivo o distinto tamaño de canvas (al estar normalizada por SSE_max).

¿Qué es un individuo en este problema? ¿Cuáles serían sus genes?

Ver “Estructura del cromosoma” más arriba: un individuo es un vector `float32` de 11
* budget genes, agrupados de a 11 por triángulo
(X1,Y1,X2,Y2,X3,Y3,R,G,B,A,ACTIVE), todos normalizados a $[0,1]$.

¿Qué es el fitness en este problema?

Ver “Función de fitness” más arriba: el RMSE normalizado e invertido, con un piso estrictamente positivo para que la selección tipo ruleta esté bien definida, y sin penalización por cantidad de triángulos activos (decisión A-02).

¿Cómo podría mutar un individuo?

Implementamos las cuatro variantes de mutación que exige la cátedra sobre cuántos genes toca cada evento de mutación: **gen** (un único gen cambia con probabilidad P_m), **multigen limitada** (una cantidad aleatoria de genes en $[1, M]$), **multigen uniforme** (cada gen, independientemente, tiene probabilidad P_m de mutar) y **completa** (con probabilidad P_m , mutan absolutamente todos los genes del individuo). Sobre esas cuatro variantes corren dos calendarios (schedules) de intensidad: **uniforme** (la magnitud del paso de mutación no cambia con la generación) y **no uniforme / Michalewicz** (la magnitud decae a medida que avanza la corrida, favoreciendo exploración amplia al principio y ajuste fino al final).

¿Cómo podría cruzar individuos para obtener descendencia? ¿Esa cruce me genera descendientes con buenas probabilidades de obtener un mejor individuo?

Implementamos los cuatro métodos de cruce de la cátedra: **un punto**, **dos puntos**, **uniforme** y **anular**, cada uno configurable para cortar en un límite de gen o en un límite de triángulo completo (boundary: "gene" | "triangle").

La segunda mitad de la pregunta merece una respuesta honesta, no una vendida: medimos explícitamente si la cruce posicional (un punto) mejora al individuo, con un control dedicado (ver “Resultados, no errores” más abajo), y la respuesta medida es que **no de manera confiable** sobre esta representación. Un swap posicional simple entre dos padres produce, en promedio, una diferencia de 0.56 por píxel (sobre 255) frente al render original — es decir, la cruce está reordenando la composición visual de los triángulos casi tanto como una mutación fuerte, porque no hay ninguna convención que garantice que “el triángulo en la posición 3 del padre A” y “el triángulo en la posición 3 del padre B” jueguen un rol comparable en la imagen final, y el orden de dibujado (z-order) no es conmutativo entre dos genomas con orígenes distintos.

¿Cómo sería la versión más simple de esto?

Ya la construimos y la llamamos explícitamente “Slice 0” (Fase 1): sin ningún operador de AG, se genera una única población aleatoria, se evalúa una vez contra la imagen objetivo y se escribe el resultado — sin selección, sin cruce, sin mutación, sin bucle generacional. Fue la manera de probar el genoma, el rasterizador y la función de fitness de punta a punta antes de construir ningún operador. El siguiente escalón real de complejidad fue “Slice 1” (Fase 2): exactamente un operador por familia (elite / cruce de un punto / mutación de gen / supervivencia aditiva con $K=N$) corriendo en bucle hasta superar fitness 0.97 sobre la bandera de referencia — la primera corrida que realmente es “un AG”, en su forma mínima.

¿Qué tipo de imagen, y sobre todo cómo afecta la cantidad de triángulos a la performance si quiero evaluar rápidamente mi motor de AG?

Usamos imágenes deliberadamente simples: una bandera de bandas planas, una silueta y un pictograma — los tres tipos que el propio enunciado recomienda como punto de partida, generados con primitivas de Pillow (sin depender de ninguna imagen de origen externo). Medimos primero la performance del renderizador antes de tocar el AG: a 128×128 de canvas y 50 individuos, el motor alcanza aproximadamente 82 generaciones por segundo. El tamaño de canvas es la palanca más barata que existe para acelerar la evaluación: un canvas de 64×64 mide 8.6 veces más rápido que uno de 256×256 , sin cambiar ni una línea de código, solo un parámetro. Eso hace que reducir el canvas — no la cantidad de triángulos — sea la primera palanca a tocar cuando se quiere iterar rápido sobre el motor.

¿Alcanza implementar PARCIALMENTE los requerimientos de este trabajo práctico para evaluar mi motor de AG?

Sí, y de hecho fue exactamente nuestra estrategia de desarrollo. “Slice 0” y “Slice 1” (ver PQ6) son implementaciones deliberadamente parciales: Slice 0 no tiene ningún operador de AG y Slice 1 tiene apenas un operador por familia, y ambas se usaron para validar el motor (rasterizador, función de fitness, contador de renders, esquema de eventos) mucho antes de que existiera la matriz completa de operadores. Esa fue la mitad “evaluar el motor” de la pregunta. La otra mitad — el entregable final que se presenta acá — completó después la matriz entera: los seis métodos de selección, ambas estrategias de supervivencia, los cuatro cruces y las cuatro variantes de mutación, corridos sobre 75 configuraciones distintas. Es decir, la implementación parcial fue una etapa deliberada del camino, no un sustituto del entregable completo.

Resultados, no errores

Dos hallazgos de este proyecto podrían parecer, a primera vista, defectos del motor. No lo son: están medidos, son reproducibles y están instrumentados a propósito porque el enunciado y el material de cátedra los anticipan.

La cruza destructiva

Ya lo adelantamos en PQ5: un swap posicional simple $[A, B] \rightarrow [B, A]$ sobre esta representación produce, medido en cinco semillas, una diferencia media de **0.56 por píxel** (sobre 255) frente al render sin cruzar. La causa no es un bug de implementación sino una propiedad estructural de la representación: no existe ninguna convención que alinee “qué representa el triángulo en la posición i ” entre dos padres distintos (competing conventions), y el orden de dibujado de los triángulos no es conmutativo — dibujar el triángulo A antes que el B no da el mismo resultado visual que dibujar B antes que A cuando ambos son translúcidos y se superponen.

../plots/fig_crossover_control.png compara la cruza base contra un control de solo-mutación al mismo presupuesto de renders.

crossover-destructiveness prediction: baseline crossover vs a mutation-only coi

El problema de la ruleta

Las diapositivas de la cátedra plantean directamente el siguiente ejercicio:

“Implementamos la selección RULETA en nuestro AG. Si en una población inicial, la diferencia entre el mejor individuo y el resto es de algunos órdenes de magnitud (sin ser buena solución), esto llevará a pérdida de diversidad y a la convergencia prematura. ¿Por qué? ¿Cómo lo resolverías?”

Sucede porque la ruleta asigna probabilidad de selección proporcional al fitness crudo: un único individuo “superindividuo” que por azar arrancó varios órdenes de magnitud mejor que el resto acapara casi toda la masa de probabilidad de la ruleta, aunque esté lejos de ser una buena solución en términos absolutos. Eso colapsa la diversidad genética de la población en pocas generaciones, antes de que la selección haya tenido oportunidad de explorar estructura real — exactamente la definición de convergencia prematura que da la cátedra. La solución es desacoplar la probabilidad de selección de la magnitud absoluta del fitness: **ranking** (que usa $f'(i) = (N - \text{rank}(i))/N$ en vez del fitness crudo) o **Boltzmann** (que atempera la selección con una temperatura que decae en el tiempo) logran exactamente eso.

../plots/fig_selection_diversity.png muestra, con nuestra propia matriz de selección, el colapso de diversidad bajo selección de alta presión frente a una buena solución

Matriz de operadores

La matriz de experimentos que respalda esta presentación tiene **3 brazos** (selección, con 7 métodos: elite, ruleta, universal, ranking, Boltzmann y ambas variantes de torneo; supervivencia K/N, con 6 combinaciones de razón $K/N \times$ estrategia; y honestidad de la cruce, con 2 celdas), **15 celdas** en total, **5 semillas por celda**, para **75 corridas** totales — barridas de a un factor por vez (one-factor-at-a-time) sobre `configs/baseline.json`. Esto **no** es el cruce completo de todos los factores contra todos los factores (que hubiera requerido del orden de 720 corridas): decidimos deliberadamente no correr ese cruce completo, porque el diseño one-factor-at-a-time aísla el efecto de cada operador sin confundirlo con el resto, a una fracción del costo computacional y de análisis. `../plots/fig_selection_fitness.png` muestra que las diferencias de presión de selección son reales y visibles, al mismo presupuesto de renders, en los siete métodos registrados.

pressure differences are real and visible at equal render budget across all 7 m

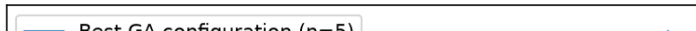


Comparación contra el (1+1) hill climber

Como punto de comparación honesto, implementamos también un **(1+1) hill climber** — una reproducción fiel del estilo EvoLisa: población de un único individuo, sin cruza, que acepta una mutación solo si mejora estrictamente el fitness actual, reutilizando el mismo Evaluator y el mismo operador de mutación configurado que usa el AG. Lo llamamos, en todo momento, “el hill climber (1+1)” — nunca “AG base” ni ningún nombre que sugiera que pertenece a la familia de los algoritmos genéticos, porque no lo es: no tiene población, no tiene cruza y no tiene selección.

../plots/fig_hillclimber_comparison.png compara el hill climber (1+1) contra la mejor configuración de AG de nuestra propia matriz de selección, **al mismo presupuesto de renders** (la curva de menor presupuesto de las dos determina el corte de ambas, para que la comparación nunca se extienda más allá de lo que el más corto efectivamente alcanzó). Reportamos el resultado tal como salió, sin recortar ni suavizar ningún tramo en el que el hill climber iguale o supere al AG.

◻ (1+1) hill climber vs the best-performing GA configuration at equal render |
(GA n=5, hill climber n=1)



Conclusiones

- ▶ Construimos un motor de Algoritmos Genéticos completo y hecho a mano — seis métodos de selección, ambas estrategias de supervivencia, los cuatro cruces y las cuatro variantes de mutación — sobre un cromosoma de longitud fija con flag de actividad, sin ninguna librería de AG.
- ▶ La comparación cuantitativa (75 corridas, 5 semillas por celda, presupuesto de renders equiparado) muestra diferencias reales y medibles entre métodos de selección, y confirma el comportamiento esperado de ambas estrategias de supervivencia bajo $K > N$.
- ▶ Dos hallazgos que podrían parecer errores — la cruza posicional destructiva y la curva no monótona de la supervivencia exclusiva — están instrumentados y reportados como comportamiento esperado del diseño, respaldados por su propia figura.
- ▶ El hill climber (1+1) es un punto de comparación honesto, no un espantapájaros: se reporta el resultado real de la comparación, gane quien gane.
- ▶ Limitación abierta: la matriz de 75 corridas publicada corre sobre un fitness ajustado antes de que se corrigiera un bug de acumulación float32 en el cálculo de SSE (Fase 1) y antes del ajuste de probabilidad de mutación de la Fase 2/3; el